

PRAKTIKUM
SISTEM BASIS DATA

Nama	Wesley Frederick Oh	No. Modul	6
NPM	2306202763	Tipe	Case Study

1. Regex

- Regex email: `^[a-zA-Z0-9]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$`
- Regex password: `^(?=.*[0-9])(?=.*[#!@%&*~]).{8,}$`

Bagian kode implementasi:

- register

```
exports.register = async function register(req, res){
  //sesuai TP
  const emailRegex = /^[a-zA-Z0-9]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;
  const passwordRegex = /^(?=.*[0-9])(?=.*[#!@%&*~]).{8,}$/;

  //bcrypt sesuai daste?
  const saltRounds = 10;

  if (!req.query.email || !req.query.password || !req.query.name) {
    return res.status(400).json({ message: 'Tidak boleh kosong!' });
  }

  if(req.query.email.match(emailRegex) == null ||
req.query.password.match(passwordRegex) == null){
    return res.status(400).json({ success: false, message: 'Regex tidak terpenuhi' });
  }

  const hashpassword = await bcrypt.hash(req.query.password, saltRounds);

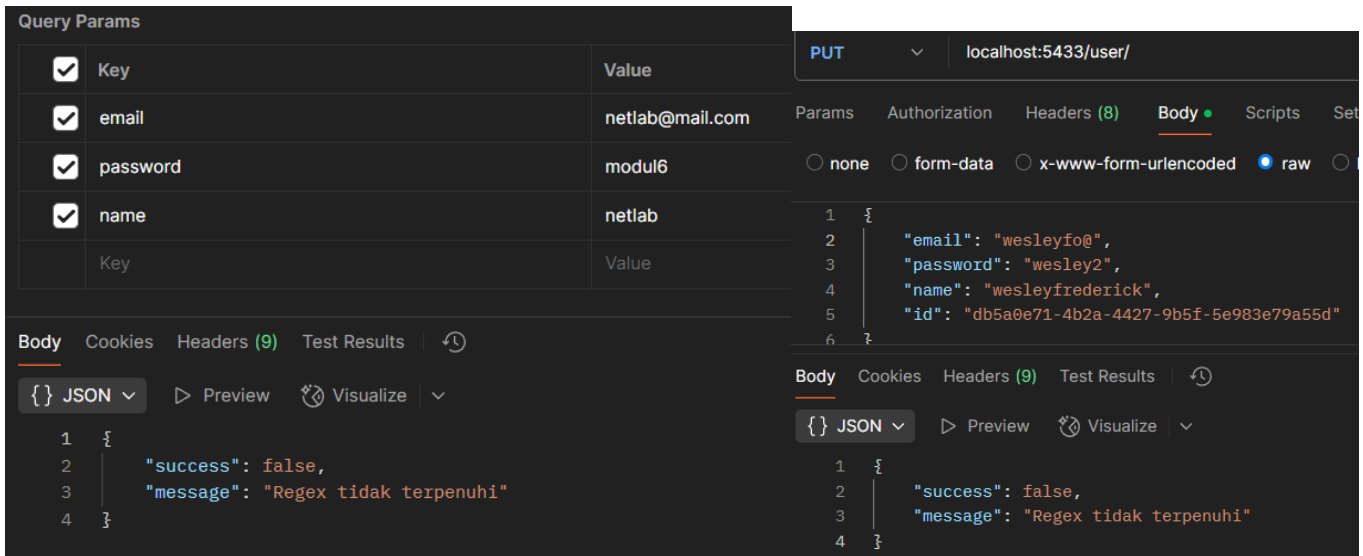
  try {
    const result = await pg.query(
      'INSERT INTO users(name, email, password) VALUES($1, $2, $3) RETURNING *',
      [req.query.name, req.query.email, hashpassword]
    );
    res.status(201).json({success: true, payload: result.rows[0]});
  } catch (error) {
    console.error('Error inserting user:', error);
    res.status(500).json({ message: 'Fail to register' });
  }
}
```

```
}  
}
```

- update

```
exports.updateUser = async function updateUser(req, res) {  
  try {  
    const { email, password, name, id } = req.body;  
    const emailRegex = /^[a-zA-Z0-9]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;  
    const passwordRegex = /^(?=.*[0-9])(?=.*[!@#$%^&*~]).{8,}$/  
  
    if (!email || !password || !name) {  
      return res.status(400).json({ message: 'Tidak boleh kosong!' });  
    }  
  
    if(email.match(emailRegex) == null || password.match(passwordRegex) == null){  
      return res.status(400).json({ success: false, message: 'Regex tidak terpenuhi'  
});  
    }  
  
    const saltRounds = 10;  
    const hashedpassword = await bcrypt.hash(password, saltRounds);  
  
    const response = await pg.query("UPDATE users SET email = $1, password = $2, name  
= $3 WHERE id = $4", [email, hashedpassword, name, id]);  
  
    //res.json(name, address);  
  
    if(response.rowCount === 0){  
      return res.status(404).json({message: "User not found"});  
    }  
  
    const cariuser = await pg.query("SELECT * FROM users WHERE id = $1", [id]);  
  
    res.status(200).json({success:true, message: "User data updated successfully",  
payload: cariuser.rows[0]});  
  } catch (error) {  
    res.status(500).json({message: "fail to update user"});  
  }  
}
```

Kalau tidak memenuhi regex (misal karena password < 8 karakter atau email salah syntax di belakang @):



Contoh di atas tidak memenuhi regex karena password < 8 karakter dan tidak ada special karakter.

2. Hash

Untuk screenshot/cuplikan kode sama seperti nomor 1 karena berada di function yang sama.

Hasil screenshot database (lihat bagian password di entry 3):

```
express_wesley=> select * from users;
+-----+-----+-----+-----+-----+
| id | balance | name | email | password |
+-----+-----+-----+-----+-----+
| 9991d8bb-d51d-4fef-bb36-1517efb2b8f0 | 0 | netlab | netlab@mail.com | modul6 |
| 2fcb1d8e-cbc4-4727-bc65-ca4c30426320 | 0 | netlab | netlab@gmail.com | modul611### |
| db5a0e71-4b2a-4427-9b5f-5e983e79a55d | 0 | wesleyfrederickoh | wesleyfo@gmail.com | $2b$10$id5Ut53cXhK9N7V6XhGI/0BZym9gGLWR |
| as4cFlsY0AWPev0/CdS9m | 0 | 2025-03-19 03:43:12.28274 |  |  |
(3 rows)
```

note: entry terakhir adalah yang final, 2 entry di atasnya itu adalah testing saja.

3. Compare Password

```
exports.login = async function login(req, res){
  if (!req.query.email || !req.query.password) {
    return res.status(400).json({ message: 'Tidak boleh kosong!' });
  }

  try {
    const cariuser = await pg.query("SELECT * FROM users WHERE email = $1",
[req.query.email]);

    if(cariuser.rowCount == 0){
      return res.status(400).json({ message: 'Akun tidak ditemukan.' });
    }
  }
}
```

```
const user = cariuser.rows[0];

const isPasswordValid = await bcrypt.compare(req.query.password, user.password);

if(!isPasswordValid){
    return res.status(400).json({ success: false, message: 'Password salah' });
}

return res.status(201).json({success: true, message: 'Login sukses', payload:
user});
} catch (error) {
    return res.status(500).json({ success: false, message: 'Gagal login', payload:
null});
}
}
```

4. CORS

```
const express = require("express");
const bodyParser = require("body-parser");
const dotenv = require("dotenv");

const rute = require("../src/routes");
const user_route = require("../src/userroute");
const item_route = require("../CS/src/itemroute");

const cors = require("cors");
dotenv.config();

const port = process.env.PORT || 5433; // Isi nomor port di sini;
const app = express();

var corsOption = {
    //origin: 'http://localhost:5433/',
    origin: 'http://os.netlab.com/',
    methods: ['GET', 'POST', 'PUT', 'DELETE'],
    preflightContinue: false,
    optionsSuccessStatus: 204
}

app.use(cors(corsOption));
app.use(bodyParser.json());
app.use(express.urlencoded({ extended: false }));

app.use("/store", cors(corsOption), rute);
```

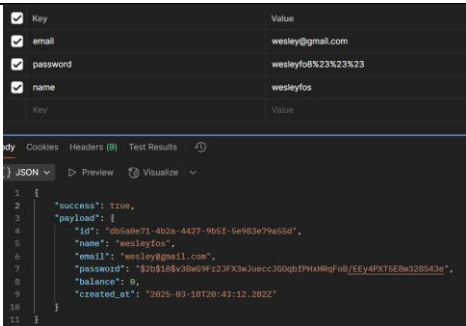
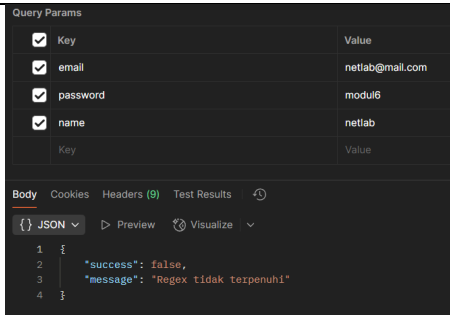
```
app.use("/user", cors(corsOption), user_route);
app.use("/item", cors(corsOption), item_route);

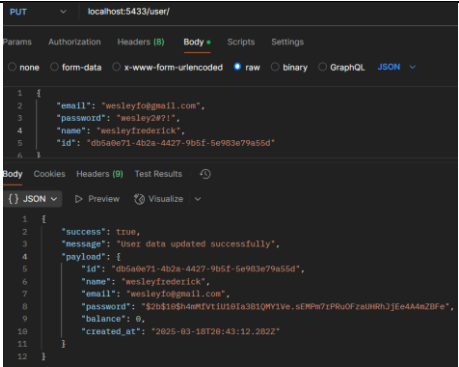
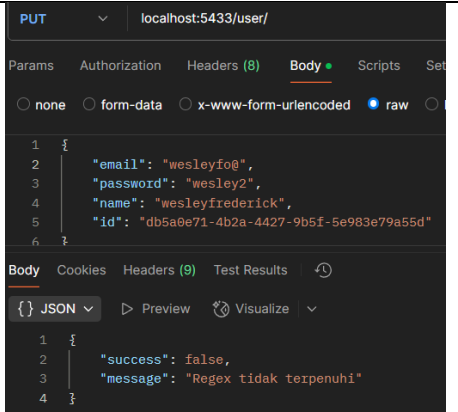
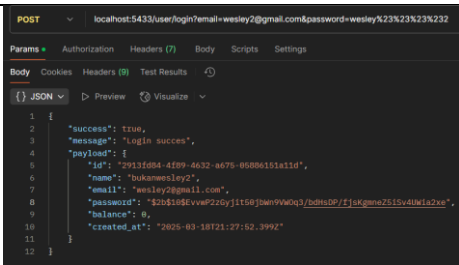
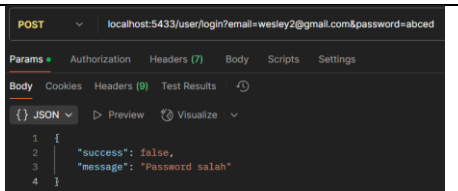
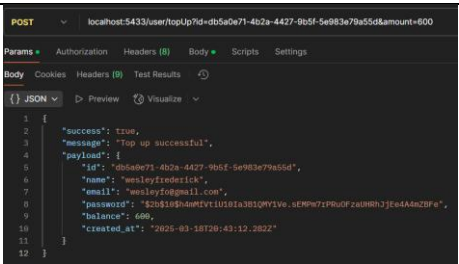
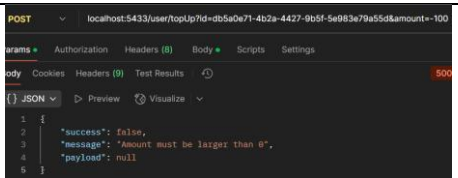
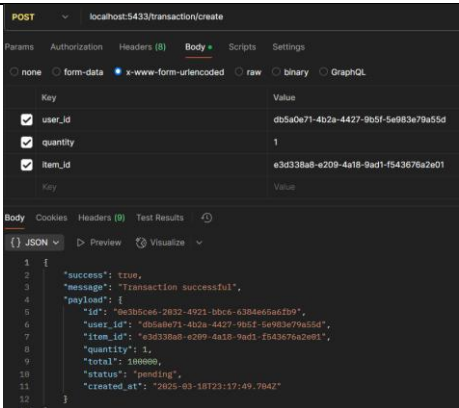
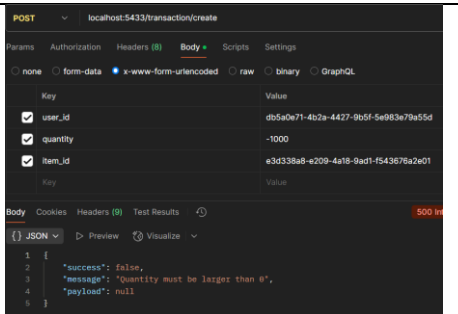
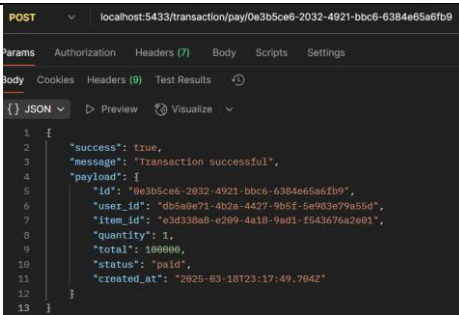
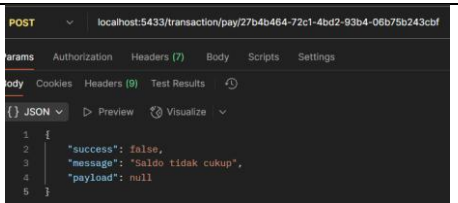
app.listen(port, () => {
  console.log(`Running on port ${port}!`);
});
```

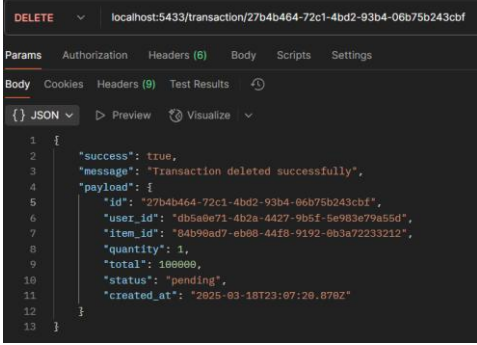
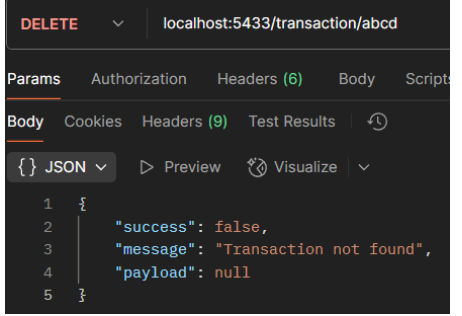
5. Memasukkan query

```
express_wesley=> CREATE type transaction_status AS ENUM ('pending', 'paid');
CREATE TYPE
express_wesley=>
express_wesley=> CREATE TABLE IF NOT EXISTS transactions(
express_wesley(>   id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
express_wesley(>   user_id UUID NOT NULL,
express_wesley(>   item_id UUID NOT NULL,
express_wesley(>   quantity INT NOT NULL,
express_wesley(>   total INT NOT NULL,
express_wesley(>   status transaction_status DEFAULT 'pending',
express_wesley(>   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
express_wesley(>   FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
express_wesley(>   FOREIGN KEY (item_id) REFERENCES items(id) ON DELETE CASCADE
express_wesley(> );
CREATE TABLE
express_wesley=> |
```

6. Screenshot Response

Method	Endpoint	Success	Failed
POST	/user/register		

PUT	/user		
POST	/user/login		
POST	/user/topUp		
POST	/transaction/create		
POST	/transaction/pay		

DELETE	/transaction/:id		
---------------	-------------------------	--	---